

C++ Programming: Systems And Programming Concepts

Chapter 3 Advanced Structure (Record)

Records/Structure (struct)

Recap:

- Struct: collection of a fixed number of components, accessed by a **single name**
- Components may be of different types
- *Components* of a struct are called the **members**
- **struct** is a reserved word

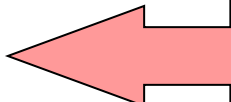
Records (struct)

- The general syntax of **struct** is:

```
struct structName
{
    DataType 1 Identifier 1;
    DataType 2 Identifier 2;
    .
    .
    .
    DataType n Identifier n;
};
```

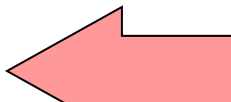
Records (structs) - Example

```
struct studentType
{
    char firstName[10];
    char lastName[10];
    char courseGrade;
    int testScore;
    int programmingScore;
    double GPA;
};
```



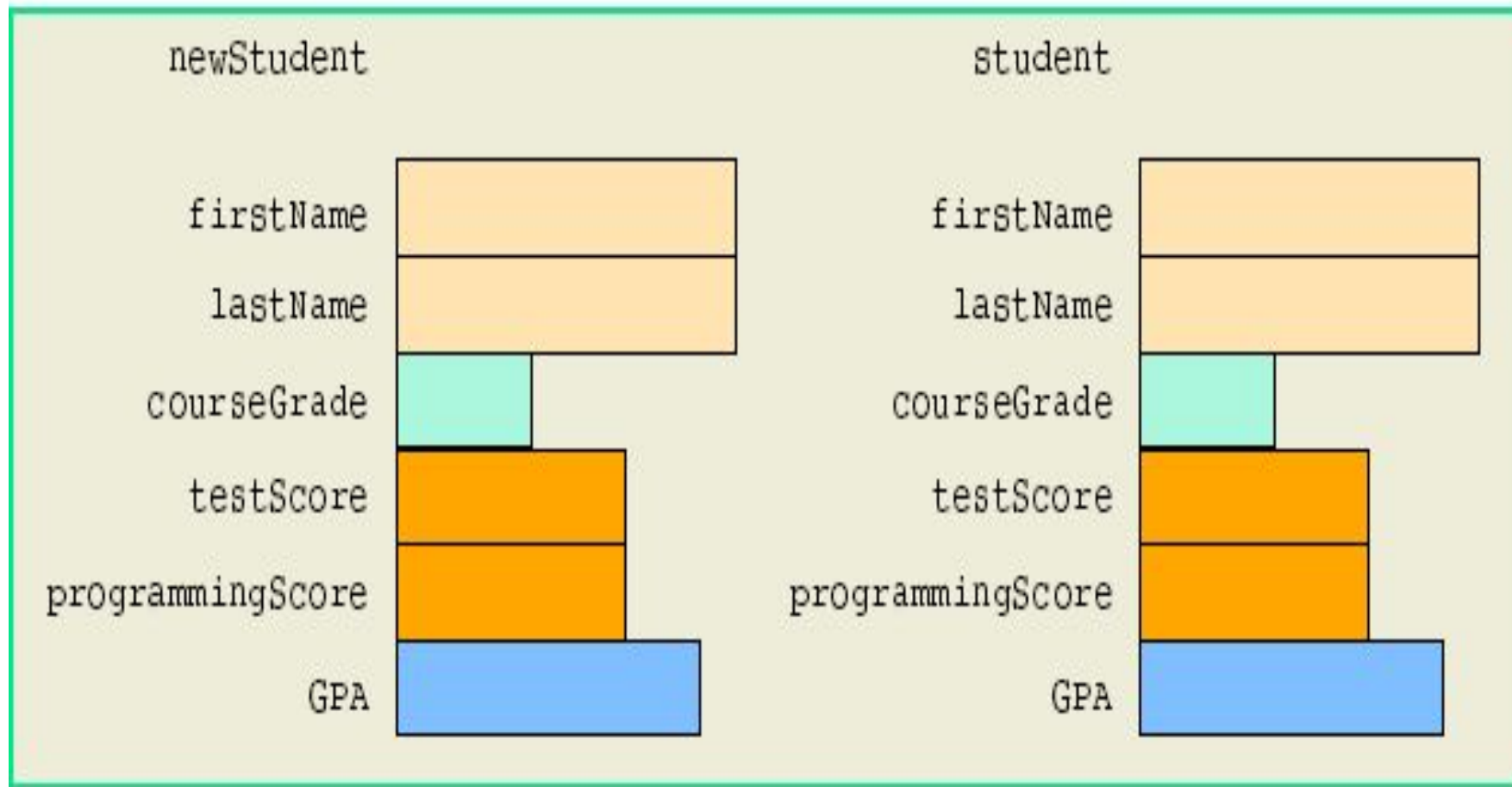
**Define a
Structure /
Record**

```
studentType newStudent, student;
```



**Declare 2
Structure
variables**

Records (structs) - Example



`struct newStudent and student`

Accessing struct Members

- The syntax for accessing a struct member is:

structVariable_Name.member_Name

- The dot (.) is an operator, called the member access operator

Accessing struct Members - Example

```
// Assign "Dennis" to newStudent first name  
strcpy (newStudent.firstName, "Dennis");
```

```
// Assign grade 'B' to newStudent course grade  
newStudent.courseGrade = 'B';
```

```
// Assign 100 to newStudent test score  
newStudent.testScore = 100;
```

```
// Read a value to newStudent GPA  
cin >> newStudent.GPA;
```

```
// Display the newStudent last name  
cout << newStudent.lastName;
```

Direct
assignment
for string
(array),
newStudent.
firstName =
"Dennis"; Is
NOT
allowed

Initializing struct members

// Declare & initialize newStudent structure

studentType newStudent = {"Lily", "Ng", 'A', 70, 88, 3.20};

OR

// Initialize newStudent structure members by using
assignment statement '='

strcpy(newStudent.*firstName*, "Lily");

strcpy(newStudent.*lastName*, "Ng");

newStudent.*courseGrade* = 'A';

newStudent.*testScore* = 70;

newStudent.*programmingScore* = 88;

newStudent.*GPA* = 3.20;

Assignment of struct variable

- Value of one struct variable can be assigned to another struct variable of the same type using an assignment '=' statement
- The statement:

student = newStudent;

copies the contents of *newStudent* into *student*

Assignment of struct variable

student = newStudent;

< same as >

student.firstName = newStudent.firstName;

student.lastName = newStudent.lastName;

student.courseGrade = newStudent.courseGrade;

student.testScore= newStudent. testScore;

**student.programmingScore=newStudent.programming
Score;**

student.GPA = newStudent. GPA;

Input/Output

- Data in a struct variable must be read one member at a time

cin>>newStudent.firstName;

- The contents of a struct variable must be written one member at a time

cout << newStudent.testScore;

Arrays in structs

- Array as one of the members in the structs
- E.g.

```
struct listType
```

```
{
```

```
    int listCode[1000]; //array containing the list
```

```
    int listLength;      //length of the list
```

```
};
```

```
listType intList;      // struct variable declaration
```

Arrays in structs (continue)

intList

listCode

listCode[0]
listCode[1]
listCode[2]
:
:
listCode[999]



listLength



Arrays in structs (continue)

- Consider the following statements:

// assign 0 to member listLength

```
intList.listLength = 0;
```

// assign 12 to the first element of array member listCode

```
intList.listCode[0] = 12;
```

// increase the 2nd element of array member listCode by 5

```
intList.listCode[1] += 5;
```

// user inputs all the values for array member listCode

```
for ( i = 0 ; i < 1000; i++ )
```

```
    cin >> intList.listCode[i];
```

// display the value of 11th element of the member listCode

```
cout << intList.listCode[10];
```

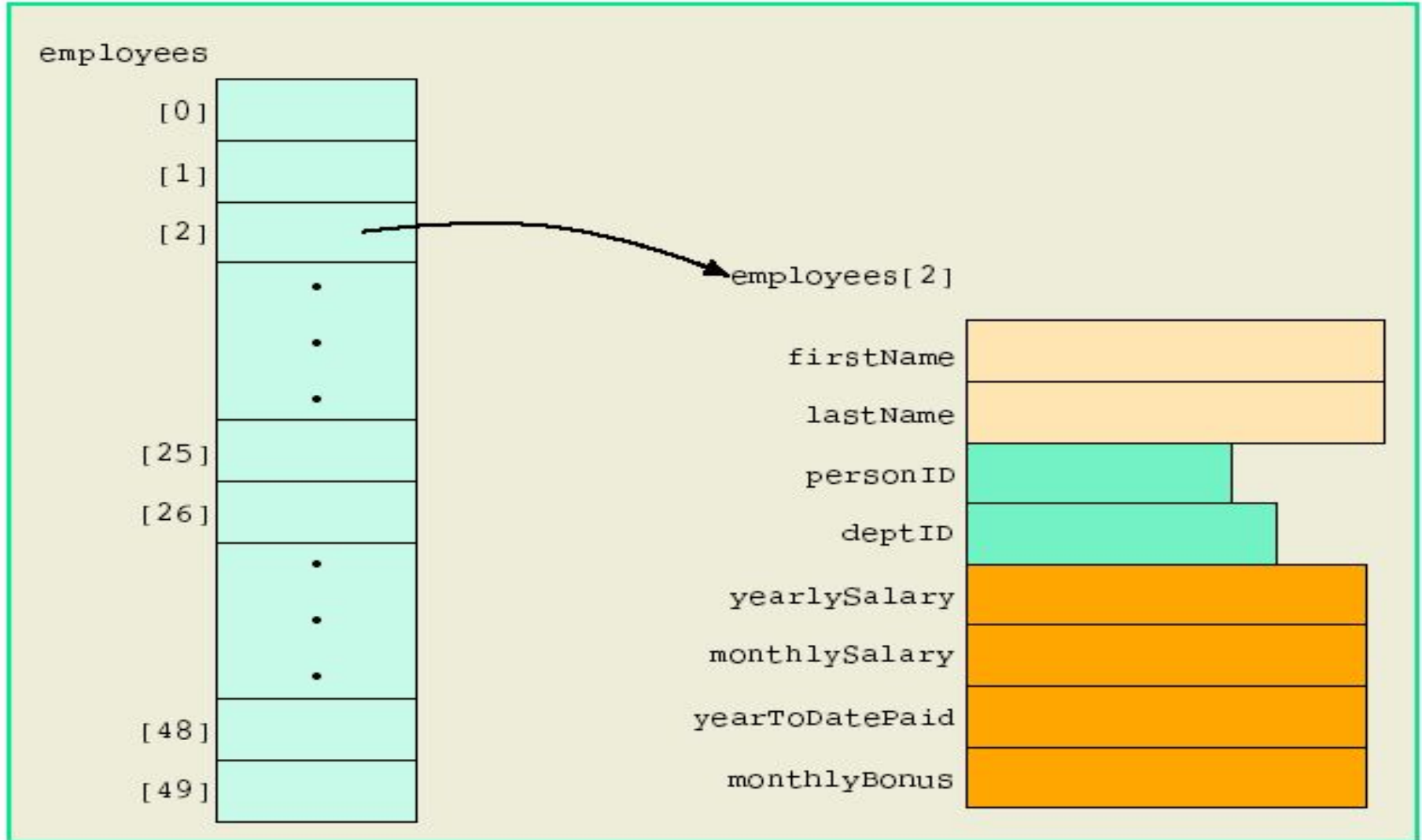
Arrays in structs - Exercise

- Create a structure with the following members:
 - Student name of string type
 - Year of study of integer type
 - Array of 5 coursework marks of float type
- Prompt user to enter the above information
- Calculate and display the above info with total and average coursework marks obtained by the student.

Structs in arrays

- Every array element is a struct type
- E.g. struct employeeType {
 char firstName[10];
 char lastName[10];
 int personID;
 char deptID;
 double yearlySalary;
 double monthlySalary;
 double yearToDatePaid;
 double monthlyBonus;
};

employeeType **employee[50];** // struct variable
declaration



Array of employees

Structs in arrays (continue)

- Consider the following statements:

```
// assign "IT999" to the dept ID of 1st employee
```

```
strcpy(employee[0].deptID, "IT999");
```

```
// increase the 20th employee monthly salary by 1200
```

```
employee[19].monthlySalary += 1200.00;
```

```
// assign year to date paid of 84500.00 to the last employee
```

```
employee[49].yearToDatePaid = 84500.00;
```

```
// user inputs monthly bonus for all 50 employees
```

```
for ( i = 0 ; i < 50; i++ )
```

```
    cin >> employee[i].monthlyBonus;
```

```
// display the 5th letter of first name of the 36th employee
```

```
cout << employee[35].firstName[4];
```

Structs in arrays - Exercise

- For managing the employee attendance of a department, create a structure *Employee* of the following members:
 - emplID of integer type
 - name of string type
 - daysPresent of integer type
- Declare an array of structure called Employees of size 3 and prompt the user to enter 3 sets of employee info above into the array
- Display all the employee details along with the percentage of attendance. Assume they have to work 24 days in a month.

Structs in structs

- Struct variable is found/nested in struct variable

```
struct nameType {  
    char first[10];  
    char middle[10];  
    char last[10];  
}
```

```
struct addressType {  
    char address1[20];  
    char address2[20];  
    char city[15];  
    char state[12];  
    int zip;  
}
```

```
struct dateType {  
    int month, day,  
    year;  
}
```

```
struct contactType {  
    char phone[15];  
    char cellphone[15];  
    char fax[12];  
    char pager[12];  
    char email[30];  
}
```

Structs in structs (continue)

- After the employee's name, address, contact and date are separated into subcategories, the new employee's record is as below:

```
struct employeeType {  
    nameType name;  
    char emplID[10];  
    addressType address;  
    dateType hireDate, quitDate;  
    contactType contact;  
    char deptID[10];  
    double salary;  
}
```

employeeType newEmployee; // struct variable declaration

newEmployee		
name	first	
	middle	
	last	
empID		
address	address1	
	address2	
	city	
	state	
	zip	
hireDate	month	
	day	
	year	
quitDate	month	
	day	
	year	
contact	phone	
	cellphone	
	fax	
	pager	
	email	
deptID		
salary		

struct variable newEmployee

Structs in structs (continue)

- Consider the following statements:

// assign "John" to the first name

```
strcpy(newEmployee.name.first, "John");
```

// assign zip code 53300

```
newEmployee.address.zip = 53300;
```

// read from user the month of hiredate

```
cin >> newEmployee.hireDate.month;
```

// display the email

```
cout << newEmployee.contact.email;
```

Structs in structs - Exercise

- Create a program for a car dealer to store info about customers and the cars they own.
 - Define a structure *Car* with
 - brand and model of string type
 - yearMade of integer type
 - Define a structure *Owner* with
 - name of string type
 - ownerIC of integer type
 - car of Car structure type
 - Create an array called owner of Owner type
 - Prompt user to input 3 sets of owner info
 - Display a report showing the owner info